

# .NET 10 unit testing

TMS grading rules · pyramid · assertions · mocks · fast feedback

---

TUnit-style assert

```
[Test]
```

```
public async Task HighMidterm_ReturnsPass()
{
    await Assert.That(result)
        .IsEqualTo(LetterGrade.Pass);
}
```

# Why TMS needs fast, isolated .NET tests at the bottom of the pyramid

---

# The "broken bridge" scenario

- TMS ships; Monday the final grade column shows Fail for every course — even sections with midterms above 90%
- Bug: someone changed `if (midtermScore > 70)` to `if (midtermScore < 70)`
- Consequence: silent wrong grades until a human notices — a unit test on the rule fails in the runner in seconds

# Framework choice: xUnit vs TUnit

- xUnit: wide sample corpus; familiar to teams and many interview panels
- TUnit: source-generated discovery; strong fit for parallel runs and AOT-friendly test builds

| Feature   | xUnit                 | TUnit                                  |
|-----------|-----------------------|----------------------------------------|
| Discovery | Reflection-based      | Source-generated (compile-time)        |
| Speed     | Fast                  | Often faster at scale (team-dependent) |
| Execution | Sequential by default | Parallel by default                    |

# TUnit in .NET 10 projects

GradingRulesTests (trimmed)

```
// GradingRulesTests.cs – TUnit-style (trimmed)
namespace Tms.Tests;
public class GradingRulesTests {
    [Test]

    public async Task CalculateFinalGrade_HighMidterm_ReturnsPass() {

        var course = new Course { MidtermScore = 85.0 };

        var result = GradingService.CalculateFinalGrade(course);

        await Assert.That(result).IsEqualTo(LetterGrade.Pass);

    }
}
```

- TUnit: source generation for compile-time test discovery in typical setups — verify behaviour against current TUnit docs before cohort install day

# Testing pyramid economics

| Metric            | Unit         | Integration      | E2E             |
|-------------------|--------------|------------------|-----------------|
| Execution time    | ~5ms         | ~100ms           | ~3 seconds      |
| Maintenance cost  | Low          | Medium           | High            |
| Failure precision | Exact method | Pipeline area    | Something broke |
| Environment needs | None         | In-memory server | Real browser    |

- Decision rule: 500 unit tests in ~3s -> run on every save; 10 E2E in ~30s -> run before commit — slow suites get skipped in real teams

# Readable assertions (FluentAssertions)

FluentAssertions (trimmed)

```
// Assertions should read as behaviour (trimmed)
```

```
[Test]
```

```
public void GetCourse_ById_ShouldReturnDetailedObject() {
```

```
    var result = _service.GetById(1);
```

```
    result.Should().NotBeNull()
```

```
        .And.BeOfType<CourseDto>()
```

```
        .And.Match<CourseDto>(c => c.Title == "Web API Workshop");
```

```
}
```

# Mocking the boundary (Moq / NSubstitute)

NSubstitute + Moq (trimmed)

```
// CourseServiceTests.cs — test the service, not the DB
var mockRepo = Substitute.For<ICourseRepository>();

mockRepo.GetById(1).Returns(new Course { Id = 1, Title = "Test" });

var service = new CourseService(mockRepo);

var result = service.GetCourseTitle(1);

result.Should().Be("Test");
```

- Swap the repository for a test double at the boundary — keep TMS domain logic under real assertions
- Moq: `mockRepo.Setup(r => r.GetById(1)).Returns(new Course { Id = 1, Title: "Test" });` — same boundary, different syntax



# When to mock — decision rule

| Dependency type       | Mock it? | Why                                       |
|-----------------------|----------|-------------------------------------------|
| Database / repository | Yes      | Slow, stateful, connection-dependent      |
| HTTP API client       | Yes      | Unreliable, slow, external                |
| File system           | Yes      | Side effects, non-deterministic           |
| The class under test  | Never    | You would be testing the mock             |
| Pure value objects    | Never    | No side effects                           |
| Configuration         | Maybe    | Mock if disk-backed; inject values if not |

# Common mistakes — Session 1

| Mistake                              | Symptom                                | Fix                                                 |
|--------------------------------------|----------------------------------------|-----------------------------------------------------|
| Missing await on TUnit Assert.That() | Test passes without checking assertion | await Assert.That(result).IsEqualTo(expected);      |
| Shared state between tests           | Pass alone, fail together              | New instance per test — avoid static mutable fields |
| Mocking the class under test         | Test verifies the mock                 | Mock ICourseRepository / APIs — not CourseService   |
| Test name: Test1 or GradingTest      | No context in CI output                | Method_Condition_ExpectedResult                     |
| No boundary values in [Theory]       | Happy path only                        | Add negative, zero, max, overflow inputs            |

# Carry into the lab

- Isolation: shared static state or shared files -> order-dependent failures
- Deterministic data: hide `DateTime.UtcNow` (and similar) behind `TimeProvider` / `IClock` in logic you test
- Fast feedback: pure unit suites stay seconds, not minutes — long runs get skipped in real teams

# Next: Session 2 — Angular 22 zoneless + Playwright

---

Vitest, TestBed, TDD, Playwright E2E — TMS frontend stays honest.